WEB APPLICATION SECURITY ASSESSMENT

# 웹 취약점 진단 보고서

PHP + MariaDB **Admin Lab** — 소스코드 기반 정적분석 및 동적 점검 결과

**대상 시스템** : Admin Lab (관리자 대시보드)

**진단 범위** : 전체 소스 20개 파일 + 실구동 사이트

**기술 스택** : PHP 8.3 / MariaDB / PDO

**진단 방식** : 소스코드 리뷰 + 블랙박스 검증

**진단 기준** : codespaces-blank-main (최신본)

**작성일** : 2026-10-27

**총 발견 건수** : 11건 (치명 3 · 높음 4 · 중간 3 · 낮음 1)

**환경** : 로컬 격리 실습 환경 (VMware/Codespaces)

## 1. 요약 (Executive Summary)

본 진단은 **Admin Lab** 관리자 시스템의 전체 소스코드를 분석하고, 실제 구동 중인 사이트에서 핵심 취약점을 재현·검증하는 방식으로 수행되었습니다. 그 결과 **인증 우회(SQL Injection)**, **임의 파일 업로드를 통한 원격 코드 실행(RCE)**, **다수 지점의 SQL 인젝션** 등 서버·데이터 전체를 장악할 수 있는 치명적 취약점이 다수 확인되었습니다.

특히 가장 위험한 것은 **최저 권한 계정(viewer)으로도 동영상 업로드 페이지에 접근해 PHP 웹셸을 올려 서버에서 임의 코드를 실행**할 수 있다는 점입니다 (업로드 기능에 권한 검사가 없음). 이 한 가지만으로 서버 완전 장악이 가능하며, 로그인 SQL 인젝션과 결합하면 비밀번호 없이도 공격이 시작됩니다. 반면 게시판(posts) 기능은 출력 이스케이프와 파라미터 바인딩이 적용되어 상대적으로 안전했습니다.

### 심각도 분포 및 발견 목록

| # | 심각도 | 취약점 | 위치 | 검증 |
|---|---|---|---|---|
| V-01 | 치명적 | 로그인 SQL 인젝션 (인증 우회) | `auth.php` | 재현됨 |
| V-02 | 치명적 | 임의 파일 업로드 → 원격 코드 실행(RCE) | `videos.php` | 재현됨 |
| V-03 | 치명적 | 다수 지점 SQL 인젝션 (동영상 기능) | `videos / video_edit / video_delete` | 코드확인 |
| V-04 | 높음 | 접근제어 미흡 (viewer가 업로드/수정/삭제) | `videos·video_edit·video_delete` | 코드확인 |
| V-05 | 높음 | 저장형 XSS (동영상 목록/수정) | `videos.php·video_edit.php` | 코드확인 |
| V-06 | 높음 | 비밀번호 평문 저장 및 비교 | `schema.sql·auth.php` | 코드확인 |

| # | 심각도 | 취약점 | 위치 | 검증 |
|---|---|---|---|---|
| V-07 | 높음 | CSRF (토큰 부재 + GET 상태변경) | `post_delete·video_delete·users` | 재현됨 |
| V-08 | 중간 | 세션 관리 취약 (고정·플래그·재생성) | `auth.php·logout.php` | 일부재현 |
| V-09 | 중간 | 보안 헤더 부재 · 평문 HTTP | 전역 | 재현됨 |
| V-10 | 중간 | 약한 기본 자격증명 · 화면 노출 | `login.php·README` | 재현됨 |
| V-11 | 낮음 | 정보 노출 (버전·DB계정·세션내 평문PW) | 응답헤더·`config·auth` | 재현됨 |

※ **안전하게 구현된 부분** — 게시판(posts) 기능은 출력 시 `htmlspecialchars()`로 이스케이프하고 조회·수정·삭제 쿼리에 PDO 파라미터 바인딩(`prepare/execute`)을 사용하여, 저장형/반사형 XSS 및 SQL 인젝션이 모두 차단되었습니다. 동일한 방식을 전체 기능으로 확장하는 것이 수정의 핵심입니다.

## 2. 상세 분석 (Findings)

### V-01 로그인 SQL 인젝션 (인증 우회) 치명적

**위치** auth.php · login_user()

**분류** CWE-89 SQL Injection | OWASP A03:2021

**설명** 로그인 검증 쿼리가 사용자 입력을 **따옴표로 직접 문자열에 삽입**합니다. 파라미터 바인딩이 없어, 공격자가 SQL 구문을 주입해 비밀번호 없이 인증을 통과하거나 DB 전체를 조회할 수 있습니다.

```
// auth.php (취약 코드)
$sql = "SELECT * FROM users WHERE username='$username' AND password='$password'";
$u = $pdo->query($sql)->fetch();   // 바인딩 없이 그대로 실행
```

**재현 (PoC)** 아이디 입력란에 아래 값, 비밀번호는 아무 값:

```
아이디: admin'-- -        → 비밀번호 조건 주석 처리, admin 로그인
아이디: ' OR 1=1-- -      → 첫 번째 사용자(admin)로 로그인
```

실제 구동 사이트에서 위 두 페이로드로 비밀번호 없이 로그인되는 것을 확인함.

**수정 방안**

- Prepared Statement(바인딩) 사용: `$pdo->prepare('SELECT * FROM users WHERE username=?')` 후 `execute([$username])`로 사용자 조회
- 조회된 해시와 `password_verify()`로 비밀번호 비교(V-06 참조). 입력값을 쿼리 구문에 직접 연결 금지

## V-02 임의 파일 업로드 → 원격 코드 실행(RCE) 치명적

**위치** `videos.php` (파일/폴더 업로드)

**분류** CWE-434 Unrestricted Upload | OWASP A01/A03

**설명** 업로드 파일의 **확장자·MIME·내용을 전혀 검증하지 않고** 원본 파일명 그대로 `uploads/`에 저장합니다. 이 디렉터리는 웹에서 직접 접근 가능하고 PHP가 실행되므로, **.php 웹셸을 올려 서버에서 임의 코드를 실행**할 수 있습니다. `accept="videos/*"`는 브라우저 힌트일 뿐 서버 방어가 아닙니다.

```
// videos.php (취약 코드)
$originalName = $files['name'][$i];
$savePath = $uploadDir . $originalName;        // 검증 없이 원본명 사용
move_uploaded_file($files['tmp_name'][$i], $savePath);
```

**재현 (PoC)**

```
# 내용이 <?php echo 7*7; ?> 인 poc.php 를 업로드 후
GET /uploads/poc.php   →   49   # PHP 실행 확인 = RCE
```

실제 구동 사이트에서 .php 업로드 후 접근 시 PHP가 실행됨을 확인(테스트 파일은 삭제 완료).

**부가 위험** 폴더 업로드(`webkitdirectory`)는 파일명에 하위 경로를 포함할 수 있고, 코드가 `dirname()`으로 디렉터리를 생성하므로 **경로 조작(Path Traversal) 기반 임의 경로 쓰기** 가능성도 있습니다.

**수정 방안**

- 확장자 **화이트리스트**(mp4/webm 등)만 허용 + 실제 파일 시그니처(매직바이트) 검증
- 저장 파일명을 서버가 **난수로 재생성**, 원본 경로·상위경로(`../`) 제거(basename 처리)
- `uploads/`에서 **PHP 실행 차단** — Apache `php_admin_flag engine off` / nginx `location ~ \.php$ { deny all; }`, 가능하면 웹 루트 밖에 저장

## V-03 다수 지점 SQL 인젝션 (동영상 기능) 치명적

**위치** video_edit.php, video_delete.php, videos.php(링크/파일 등록)

**분류** CWE-89 SQL Injection

**설명** 동영상 기능 전반이 prepare 없이 query()/exec()에 입력값을 직접 연결합니다. **숫자 id와 제목·URL·파일명** 모두 주입 지점입니다.

```
// video_edit.php / video_delete.php — $id는 GET 입력, 정수 변환 없음
$id = $_GET['id'] ?? 0;
$sql = "SELECT * FROM videos WHERE id=$id";      // 예: ?id=0 UNION SELECT ...

// videos.php — 링크 등록, 제목/URL 직접 삽입
$sql = "INSERT INTO videos (...) VALUES ('$title','$url','$url', ...)";
```

**영향** DB 내 사용자 계정·비밀번호(평문) 탈취, 데이터 위변조·삭제. 저장된 값이 나중에 출력되며 XSS로도 연결됩니다(V-05).

> **수정 방안** 모든 쿼리를 **Prepared Statement**로 전환. id는 (int) 캐스팅 후에도 반드시 바인딩. exec()/query()에 사용자 입력 문자열을 연결하지 말 것.

## V-04 접근제어 미흡 (권한 검사 누락) 높음

**위치** videos.php, video_edit.php, video_delete.php

**분류** CWE-862 Missing Authorization · OWASP A01

**설명** 게시판(post_form/post_delete)은 require_role(['admin','editor'])로 보호되지만, **동영상 기능에는 역할 검사가 없습니다**. 로그인만 되어 있으면(require_login) 됩니다. 결과적으로 **조회 전용(viewer) 계정이 동영상을 업로드·수정·삭제**할 수 있고, 업로드는 곧 RCE(V-02)이므로 **최저 권한으로 서버 장악**이 가능합니다.

```
// videos.php / video_edit.php → header.php 의 require_login()만 수행 (역할 미검사)
// video_delete.php → require 'auth.php'; 직후 require_role() 호출 없음
```

> **수정 방안** 동영상 쓰기·수정·삭제에도 require_role(['admin','editor'])를 적용하고, 서버 측에서 역할을 강제. 메뉴 숨김(header.php)은 방어가 아님.

## V-05 저장형 XSS (동영상 목록/수정) 높음

**위치** videos.php(목록), video_edit.php(입력값 반영)

**분류** CWE-79 Stored XSS | OWASP A03

**설명** 동영상 목록은 제목·파일명·등록자·메시지를 **이스케이프 없이** 출력합니다. 제목이나 링크(URL)에 스크립트를 넣어 등록하면, 해당 페이지를 보는 관리자 브라우저에서 **스크립트가 실행**됩니다(세션 쿠키가 HttpOnly가 아니므로 세션 탈취로 직결 — V-08).

```
// videos.php 목록 출력 — e() 미적용
<td> <?= $r['title'] ?> </td>
<a href="<?= $r['filename'] ?>"> <?= $r['original_name'] ?> </a>
<p class="tag"> <?= $msg ?> </p>
```

대조: posts 기능은 `<?= e($p['title']) ?>`로 이스케이프되어 XSS가 차단됨.

> **수정 방안** 모든 사용자 데이터 출력에 `e()`(`htmlspecialchars`, ENT_QUOTES) 적용. 속성값 삽입 시 따옴표 포함 인코딩, 링크는 `http/https` 스킴 화이트리스트 검증.

## V-06 비밀번호 평문 저장 및 비교 높음

**위치** schema.sql(초기데이터), auth.php(비교)

**분류** CWE-256/916 Plaintext Password | OWASP A02

**설명** 비밀번호가 DB에 **평문**으로 저장되고, 로그인도 평문 문자열 일치로 검증합니다. DB가 한 번 유출되면(V-01/V-03로 가능) 모든 계정 비밀번호가 즉시 노출됩니다.

```
// schema.sql
INSERT INTO users(...) VALUES ('admin', 'admin123', '관리자', 'admin'); ...
```

> **수정 방안** 저장 시 `password_hash($pw, PASSWORD_DEFAULT)`, 검증 시 `password_verify()` 사용. 기존 평문 비밀번호는 전면 재설정.

## V-07 CSRF (토큰 부재 + GET 상태변경) 높음

**위치** post_delete.php, video_delete.php(GET 삭제), users.php/post_form.php/video_edit.php(POST, 토큰 없음)

**분류** CWE-352 CSRF | OWASP A01

**설명** 상태를 바꾸는 요청에 **CSRF 토큰이 전혀 없고**, 삭제는 **GET**으로 동작합니다. 관리자가 로그인된 상태에서 공격자가 만든 페이지를 열기만 해도 게시물·동영상 삭제나 권한 변경이 유발될 수 있습니다.

```
// 첨부된 "CSRF 실습" 메모의 재현 방식과 동일
<img src="http://.../post_delete.php?id=2">   # 페이지 방문만으로 2번 게시물 삭제
```

실제 사이트에서 post_delete.php?id=N GET 요청만으로 삭제됨을 확인.

> **수정 방안** 모든 상태 변경을 **POST + CSRF 토큰**(요청마다 검증)으로 전환. 세션 쿠키 SameSite=Lax/Strict 설정. 삭제를 GET으로 처리하지 말 것.

## V-08 세션 관리 취약 중간

**위치** auth.php(login_user), logout.php

**분류** CWE-384 Session Fixation | CWE-1004 쿠키 플래그

**설명** ① 로그인 성공 시 session_regenerate_id()가 없어 **세션 고정(Session Fixation)** 공격에 노출됩니다. ② 세션 쿠키에 **HttpOnly/Secure/SameSite가 설정되지 않아** JavaScript(document.cookie)로 PHPSESSID가 읽혔습니다 — XSS(V-05) 발생 시 세션 탈취로 직결. ③ 세션에 사용자 레코드 전체($_SESSION['user']=$u)를 담아 평문 비밀번호까지 포함됩니다.

```
// 실측: JS에서 세션 쿠키가 그대로 노출됨 (HttpOnly 미설정)
document.cookie → "PHPSESSID=rem3fvm1jo6k3k2hmkavc8bpef"
```

> **수정 방안** 로그인 직후 session_regenerate_id(true); 쿠키 파라미터 httponly=1, secure=1, samesite=Lax 설정(session_set_cookie_params); 세션엔 id·role 등 최소 정보만 저장.

## V-09 보안 헤더 부재 · 평문 HTTP 중간

**분류** CWE-693 Protection Mechanism Failure | OWASP A05

**설명** 응답에 `Content-Security-Policy`, `X-Frame-Options`, `X-Content-Type-Options`, `Strict-Transport-Security`, `Referrer-Policy`가 **하나도 없습니다**. 또한 전 구간이 **평문 HTTP**로 동작하여 자격증명·세션이 네트워크에서 도청될 수 있습니다. 보안 헤더 부재는 클릭재킹·MIME 스니핑·XSS 방어선 부재로 이어집니다.

> **수정 방안** HTTPS 적용 및 HTTP→HTTPS 리다이렉트, HSTS 적용. 최소 `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, 적절한 `Content-Security-Policy`, `Referrer-Policy` 추가.

## V-10 약한 기본 자격증명 · 화면 노출 중간

**위치** `login.php`, `README.md`, `schema.sql`

**분류** CWE-521 Weak Credentials | CWE-200 정보노출

**설명** `admin/admin123`, `editor/editor123`, `viewer/viewer123` 등 **추측하기 쉬운 비밀번호**이며, **로그인 화면에 기본 계정이 그대로 표시**됩니다. 또한 로그인 시도 횟수 제한·계정 잠금·CAPTCHA가 없어 무차별 대입에 노출됩니다.

> **수정 방안** 운영 배포 시 기본 계정 제거/변경, 화면의 계정 안내 제거, 강력한 비밀번호 정책, 로그인 실패 기반 지연·잠금(rate limiting) 적용.

## V-11 정보 노출 낮음

**위치** 응답 헤더, `config.php`, `auth.php`

**분류** CWE-200 Information Exposure

**설명** 응답 헤더에 `X-Powered-By: PHP/8.0.30`로 **서버 버전이 노출**되었고(해당 PHP 8.0대는 지원 종료(EOL) 계열), `config.php`에 **DB 계정이 하드코딩**(`adminlab/adminlab123`)되어 있습니다. 세션에 평문 비밀번호가 포함되는 점(V-08)도 정보 노출에 해당합니다.

> **수정 방안** `expose_php = Off`로 버전 헤더 제거, 지원 중인 PHP로 업그레이드, DB 자격증명은 환경변수/설정 파일 분리 및 저장소 커밋 금지.

## 3. 공격 시나리오 (Attack Chain)

개별 취약점은 서로 연결될 때 피해가 극대화됩니다. 본 시스템에서 가장 현실적인 전체 장악 경로는 다음과 같습니다.

| 단계 | 행위 | 이용 취약점 |
|---|---|---|
| 1 | 로그인 창에 `admin'--  -` 입력 → 비밀번호 없이 관리자 로그인 (또는 viewer 계정으로 로그인) | V-01 / V-10 |
| 2 | 동영상 업로드 페이지 접근 (viewer 권한으로도 가능) | V-04 |
| 3 | PHP 웹셸(.php) 업로드 → `/uploads/shell.php` 접근으로 서버 명령 실행 | V-02 |
| 4 | DB 접근해 전 계정 평문 비밀번호 탈취, 데이터 위변조 | V-03 / V-06 |
| 5 | 관리자가 악성 링크를 열도록 유도해 권한 변경·삭제 유발(보조) | V-07 / V-05 |

**결론:** 치명적 등급의 V-01·V-02·V-03를 최우선으로 조치해야 하며, 특히 **업로드 검증(V-02)과 동영상 기능 권한 검사(V-04)**는 즉시 수정이 필요합니다.

## 4. 조치 우선순위

| 우선순위 | 조치 항목 | 대상 |
|---|---|---|
| 즉시 | 업로드 확장자 화이트리스트 + uploads/ PHP 실행 차단 | V-02 |
| 즉시 | 전 쿼리 Prepared Statement 전환(로그인 포함) | V-01, V-03 |
| 즉시 | 동영상 쓰기/수정/삭제에 역할 검사 추가 | V-04 |
| 단기 | 전 출력 이스케이프 + 비밀번호 해시화 + CSRF 토큰 | V-05, V-06, V-07 |
| 중기 | 세션 플래그/재생성, 보안 헤더, HTTPS, 기본계정 정리 | V-08~V-11 |

---

본 보고서는 로컬 격리 실습 환경(Admin Lab)을 대상으로 교육·점검 목적에 한해 수행된 진단 결과입니다. 모든 테스트용 데이터와 업로드 PoC 파일은 점검 후 삭제되었으며, 변경된 설정은 원복되었습니다. 식별된 취약점은 실제 운영 환경에 배포하기 전 반드시 조치되어야 합니다.
진단 기준 소스: `codespaces-blank-main` (최신본) · 작성일 2026-10-27